ENGINEERING BRIEF · AGENT B · FEATURE 2

Roomio — Virtual Camera & Flythrough

A second Agent-B work-stream: a first-person walk mode inside Roomio's 3D scene, a top-down camera director that authors a path through the space, and a recorder that renders a smooth flythrough video along that path. This is real-time engine work on synthetic geometry — explicitly NOT NeRF / novel-view synthesis.

THREE.JS REAL-TIME · NOT NERF · SPLINE CAMERA + VIDEO EXPORT

Repo: `~/Desktop/personal/roomio` · **Your folder:** `/camera-flythrough`
Coordination: `/shared/*.json` · **Joint log:** `roomio.txt`
Date: June 29, 2026 · **Companion to:** Detection Pipeline Brief, Room Designer v2

Contents

1 Critical Framing: This Is Not NeRF 03

2 The Interaction Model (Director + Preview) 04

3 Build Order & Feature Breakdown 05

4 Technical Approach (per feature) 06

5 Video Capture: the one tricky part 09

6 Recommended Open-Source Stack 10

7 Integration, Isolation & Coordination 11

8 Functional Requirements & Acceptance 12

9 Build Milestones 13

1. Critical Framing: This Is Not NeRF

READ THIS BEFORE ANYTHING ELSE

The request was framed as "control a camera like you can in NeRF." Roomio's scene is **not** a NeRF or a Gaussian splat — it is a synthetic Three.js scene with explicit geometry (authored walls + glTF furniture). That changes everything: moving a camera through it and capturing frames is a **solved real-time-engine task**, not a novel-view-synthesis research problem. Do **not** pull in any NeRF, Gaussian-splatting, COLMAP, or reconstruction tooling. You already have the geometry.

Why the distinction matters for scoping:

- In a NeRF, "fly the camera along a path" means rendering learned radiance for never-seen viewpoints — the hard part. In a synthetic scene, you simply reposition a `PerspectiveCamera` and the engine renders the geometry it already has. Trivial by comparison.
- Everything you need is first-party Three.js: camera controls, `CatmullRomCurve3` spline paths, and canvas frame capture. No model training, no GPU inference, no point clouds.
- This means the feature is achievable in days of engine work, not weeks of research. If you find yourself reaching for a reconstruction library, stop — you've misread the task.

ONE-LINE SCOPE

Walk the synthetic room in first person; author a camera path from a top-down director view (waypoints → spline, or record a manual walk); play the camera along that path; capture the canvas frame-by-frame into a smooth video file. All client-side Three.js.

2. The Interaction Model (Director + Preview)

> A **dual-view director's setup**, mirroring how real previs / architectural flythrough tools work. The default working view is top-down (orchestration); the user can jump into the camera's first-person POV to check framing, then pop back out to adjust.

The two views

- **Top-down director view (default).** The user looks down on the room plan and lays out the camera path: drop waypoints, drag the spline, set look-at targets, set speed. A visible camera "gizmo" shows where the recording camera is and which way it faces. This is the primary authoring surface.
- **First-person preview (jump-in).** At any moment the user can drop into the camera's actual POV to see exactly what it will record, then return to the top-down view. This is a preview of the *recording camera*, distinct from free first-person walking (below).

Two ways to author the path

- **Waypoint mode:** in the top-down view, click to drop waypoints; a Catmull-Rom spline smoothly connects them; the camera will travel along it. Drag points to reshape; adjust per-point look-at and dwell.
- **Walk-and-record mode:** enter free first-person walk, move through the room (WASD + mouse-look), and record that trajectory as the path. The recorded positions become spline control points.

Both produce the same artifact: a **camera path** (ordered control points + look-at + timing) that the playback/record stage consumes. Store it as JSON so it's reusable and shareable.

3. Build Order & Feature Breakdown

Build in this order — each stage is independently testable and unblocks the next. This matches the sequence requested: walk first, then a controllable path camera, then recording.

#	Feature	What it does	Core tech
F1	First-person walk	Walk the synthetic room in first person (WASD + mouse-look), with collision so you can't pass through walls/furniture.	PointerLockControls + collision
F2	Top-down director view	Look down on the room; see a camera gizmo; switch between this and F1's POV.	2nd camera + view toggle
F3	Waypoint path authoring	Drop/drag waypoints in top view; smooth spline through them; per-point look-at & timing.	CatmullRomCurve3 + handles
F4	Walk-and-record path	Record a manual first-person walk as path control points (alternative to F3).	Sample F1 trajectory
F5	Playback along path	Animate the recording camera along the path; preview in either view; scrub/play/pause.	getPointAt(t) + lookAt
F6	Video capture	Render the playback frame-by-frame to a smooth downloadable video.	canvas-record (WebCodecs)

SEQUENCING NOTE

F1–F2 give you a usable walk + director shell fast. F3 is the heart of the feature. F4 is an alternative authoring mode — ship F3 first, add F4 after. F5 must work (smoothly) before F6 is meaningful, because F6 just captures what F5 plays.

4. Technical Approach (per feature)

F1 — First-person walk

Use Three.js `PointerLockControls` (the standard FPS control: click to capture the pointer, mouse to look, WASD to move). For collision against walls and furniture, you already have explicit geometry — reuse the same 2D-footprint / AABB approach the main app uses for furniture placement, or cast short rays in the movement direction and block movement on hit. Keep eye height fixed (~1.6 m) so it feels like walking.

```
// F1 — pointer-lock FPS walk (sketch)
import { PointerLockControls } from 'three/addons/controls/PointerLockControls.js';
const controls = new PointerLockControls(fpCamera, renderer.domElement);
renderer.domElement.addEventListener('click', () => controls.lock());
// in animate(): move along camera-facing vector by WASD, then
// clamp against wall footprints before committing the new position.
```

F2 — Top-down director view

A second camera (orthographic or a high perspective looking straight down). A UI toggle swaps which camera renders. Draw a **camera gizmo** in the scene (a small mesh + a frustum/arrow helper) at the recording camera's current path position so the director sees where it is and where it points.

F3 — Waypoint path authoring

In the top-down view, raycast clicks onto the floor plane to drop waypoints (`Vector3`). Build a `CatmullRomCurve3` from them — it passes *through* every point (unlike Bezier), which is what users expect. Render the curve as a line/tube so it's visible. Make points draggable to reshape. Store per-point look-at target and a timing/dwell value.

```
// F3 — spline through waypoints
const curve = new THREE.CatmullRomCurve3(waypoints); // array of Vector3
const pathLine = new THREE.Line(
  new THREE.BufferGeometry().setFromPoints(curve.getPoints(200)),
  new THREE.LineBasicMaterial());
scene.add(pathLine); // visualize; update on drag
```

F4 — Walk-and-record path

While in F1 walk mode, sample the camera position at a fixed interval (e.g. every N ms or M cm of travel) into an array. On stop, decimate to a sensible number of control points (fewer points = smoother curve — over-dense points make the spline jittery) and feed them into the same `CatmullRomCurve3` as F3. Now both authoring modes converge on one path artifact.

F5 — Playback along the path

Advance a parameter `t` from 0→1 over the desired duration. Each frame: set camera position to `curve.getPointAt(t)` , and aim it using a point slightly ahead, `curve.getPointAt(t + ε)` (look-ahead gives smooth turning through bends) — or the explicit per-waypoint look-at target if set. Provide play / pause / scrub. Use `getPointAt` (arc-length parameterized), not `getPoint` , so speed is constant along the curve.

```
// F5 — drive camera along curve with smooth look-ahead
const pos = curve.getPointAt(t);
const look = curve.getPointAt(Math.min(t + 0.02, 1));
recCamera.position.copy(pos);
recCamera.lookAt(look);
```

SMOOTHNESS TIPS (FROM REAL BUILDS)

Fewer control points = less jitter. Use a small look-ahead offset for lookAt so the camera anticipates bends. If position and look-at use separate curves of different lengths, advance their `t` values at different rates so the camera doesn't "look around" too slowly. These are the exact gotchas teams hit shipping cinematic Three.js paths.

5. Video Capture — the one tricky part

DO NOT USE MEDIARECORDER FOR THIS

`MediaRecorder` / `canvas.captureStream()` records in **real time** — if a frame takes too long to render, it drops frames, producing choppy, variable-quality output. For a flythrough you want deterministic quality. Use **frame-by-frame capture** instead: render frame → capture it → advance `t` → repeat, decoupled from wall-clock time. The output is perfectly smooth regardless of render speed.

Recommended: `canvas-record` (WebCodecs)

The modern, maintained library for exactly this. It records a canvas region as MP4/WebM/etc. using the browser's hardware-accelerated WebCodecs API — reportedly 5–10× faster than older JS encoders and ~20× faster than ffmpeg.wasm. It supports a step-based recording loop that fits the deterministic capture pattern. Note WebCodecs requires a secure context (HTTPS or localhost).

```
// F6 — deterministic capture loop with canvas-record
import { Recorder } from 'canvas-record';
const recorder = new Recorder(renderer.getContext(), { extension: 'mp4', frameRate: 30 });
await recorder.start();
for (let t = 0; t <= 1; t += step) {     // step = 1 / (durationSec * fps)
  setCameraAlongPath(t);                  // from F5
  renderer.render(scene, recCamera);
  await recorder.step();                  // captures the just-rendered frame
}
await recorder.stop();                    // returns the encoded video buffer → download
```

Fallbacks & constraints

- **CCapture.js** — legacy frame-by-frame capturer (Whammy encoder); reliable for older browsers but memory-heavy for long clips (accumulates blobs in RAM) and weaker on MP4. Use only if WebCodecs is unavailable.
- **Avoid ffmpeg.wasm** in-browser just to record a canvas — over-engineered and resource-hungry. If heavy transcoding is ever needed, do it server-side.
- **Resolution cap:** WebCodecs/MP4 has an AVC max frame size (~9.4M px) — fine up to a bit beyond 4K @ 16:9. Don't target 8K.

6. Recommended Open-Source Stack

Need	Tool	Why
First-person walk	three/addons **PointerLockControls**	First-party Three.js FPS control; pointer lock + WASD; no extra dependency.
Camera path	**THREE.CatmullRomCurve3**	First-party; passes through every waypoint; `getPointAt(t)` / `getTangentAt(t)` for position + direction.
Path persistence	JSON (control points + look-at + timing)	Reusable, shareable, and the hand-off artifact between authoring and capture.
Video capture	**canvas-record** (WebCodecs)	Hardware-accelerated MP4/WebM; deterministic step-based loop; far faster than alternatives.
Legacy capture	CCapture.js	Fallback for browsers without WebCodecs; memory-heavy, MP4-weak.
If using R3F	drei helpers / use-capture	If the app is React Three Fiber, drei has camera/controls helpers; match Agent A's setup.

MATCH AGENT A'S RENDERING SETUP

Whether the app is plain Three.js or React Three Fiber determines how you mount controls and grab the canvas/renderer. Confirm Agent A's setup via `roomio.txt` before wiring F1/F6 so your code drops into the same scene rather than spinning up a second renderer.

7. Integration, Isolation & Coordination

Same rules as your detection work-stream: your code is isolated, you talk to Agent A through shared files and `roomio.txt`, and you never edit the front-end source directly.

Folder & boundaries

- All your code in `/camera-flythrough`. The only other writable places are `/shared` and `roomio.txt`.
- This feature must *render the same scene* Agent A builds — so unlike detection (a standalone service), it has a real integration seam. Resolve that seam through the contract below, not by editing A's files.

The contract surface

- **Scene access:** you need the live Three.js scene/renderer Agent A creates. Agree on a mount point: e.g. Agent A exposes the scene + renderer (or a mount callback) that your module attaches to. Define this in `/shared/scene_contract.json` and request it from A in `roomio.txt`.
- **Camera path artifact:** publish `/shared/camera_path_schema.json` (control points, per-point look-at, total duration, fps). This is what authoring writes and capture reads; Agent A can also save/load it with the design.
- **Output:** the rendered video is downloaded client-side; no server needed.

roomio.txt rules (unchanged)

- Append-only; prefix `[YYYY-MM-DD HH:MM] [AGENT-B]`.
- Re-read at the start of every work cycle; treat Agent A / human entries as high-priority.
- Log INTENT before big changes, DECISIONS (library choices, schema), and explicit `REQUEST -> AGENT-A` for the scene mount point.
- Commit only your own paths (`/camera-flythrough`, your `/shared` schemas) to avoid git collisions with Agent A.

8. Functional Requirements & Acceptance

ID	Requirement	Feature
C-1	User can walk the room in first person (WASD + mouse-look) at eye height.	F1
C-2	Walking collides with walls/furniture; cannot pass through.	F1
C-3	Top-down director view with a camera gizmo showing position + facing.	F2
C-4	One-tap toggle between top-down and the recording camera's first-person POV.	F2
C-5	User drops/drags waypoints in top view; smooth spline passes through them.	F3
C-6	Per-waypoint look-at target and timing/dwell adjustable.	F3
C-7	Alternative: record a manual first-person walk as the path.	F4
C-8	Camera plays along the path smoothly (constant speed, smooth look-ahead); play/pause/scrub.	F5
C-9	Path persists as JSON per the shared schema (save/reload/share).	F3/F5
C-10	Render the flythrough to a smooth downloadable video (frame-by-frame, not realtime).	F6
C-11	All code in `/camera-flythrough`; scene accessed via the shared contract; no front-end edits.	Isolation
C-12	Decisions/contract changes/requests logged to `roomio.txt` with timestamp + [AGENT-B].	Comms

Acceptance test (the "done" bar)

In a furnished Roomio room: enter first-person and walk around without clipping through walls; switch to top-down, drop a handful of waypoints, see a smooth spline; hit play and watch the camera glide the path (previewable in first-person POV); export a smooth MP4 of that flythrough that downloads locally. The path saves and reloads from JSON.

9. Build Milestones

Milestone	Deliverable	Definition of done
C0	Scaffold `/camera-flythrough`; confirm Agent A's renderer setup (Three vs R3F); publish path + scene contracts; log intent	Folder + schemas committed; A pinged for scene mount.
C1	F1 first-person walk + collision	Walk the room, blocked by walls/furniture.
C2	F2 top-down view + camera gizmo + toggle	Switch views; see where the camera is/points.
C3	F3 waypoint spline authoring	Drop/drag points; smooth visible path; look-at + timing.
C4	F5 playback along path	Camera glides smoothly; play/pause/scrub; previewable POV.
C5	F6 frame-by-frame video export	Smooth MP4 downloads; deterministic, not realtime.
C6	F4 walk-and-record path + JSON persistence + polish	Manual-walk authoring works; paths save/reload.

> **One-line summary.** Build a client-side Three.js flythrough feature in `/camera-flythrough`: first-person walk (PointerLockControls + collision), a top-down director that authors a CatmullRomCurve3 path (waypoints or recorded walk), smooth playback via `getPointAt` with look-ahead, and a deterministic frame-by-frame video export via `canvas-record` (WebCodecs). No NeRF. Integrate through the shared scene contract; log to roomio.txt.

End of brief · Roomio Virtual Camera & Flythrough (Agent B) · Synthetic-scene real-time capture; no reconstruction tooling.